📝 GLA UNIVERSITY — DAY 2 IN-CLASS SOCRATIC CHECKPOINT PAPER

Traditional Machine Learning & Parallel Computing using Intel oneAPI (`GLA-HPC-BCSG701`)

Candidate Name: _____ | **Roll Number:** _____
Date: _____ | **Section:** _____

Instructions to Candidates:

1. 25 Multi-Choice Checkpoint questions to be answered interactively as indicated by the instructor.
2. Select only ONE option (a, b, c, or d) for each question.

Question 01 [Module 1: Parallel Computing Principles]

Question: What is the precise architectural difference between 'Concurrency' and 'True Parallelism' in high-performance computer systems?

- **a)** Concurrency executes simultaneously on multiple physical cores; Parallelism interleaves tasks via time-slicing on a single CPU core
- **b)** Concurrency is the composition of independently executing processes managed via interleaving; Parallelism is the simultaneous physical execution of multiple computational tasks at the exact same instant across multiple physical hardware cores
- **c)** Concurrency only applies to Python, while Parallelism only applies to C++
- **d)** Concurrency requires a GPU, while Parallelism requires an FPGA

Candidate Selection: [ ]

Question 02 [Module 1: Parallel Computing Principles]

Question: Which of the following statements accurately characterizes the memory address space in Shared-Memory (SMP/NUMA) versus Distributed-Memory (Cluster) computing?

- **a)** In Distributed-Memory systems, all CPU cores across different cluster racks can directly dereference the same raw pointer address without networking
- **b)** In Shared-Memory systems, all processors access a globally shared physical address space; in Distributed-Memory systems, each node possesses private local memory requiring explicit message passing over a network interconnect to share data
- **c)** Shared-Memory systems require InfiniBand switches between CPU cores on the same motherboard
- **d)** Distributed-Memory systems cannot execute C++ applications

Candidate Selection: [ ]

Question 03 [Module 1: Parallel Computing Principles]

Question: An engineer parallelizes an ML feature extraction pipeline such that 80% is parallelized across 16 CPU cores, while 20% remains strictly sequential. What is the theoretical overall speedup achieved according to Amdahl's Law?

- **a)** 16.0x
- **b)** 4.0x
- **c)** 5.0x
- **d)** 8.0x

Candidate Selection: [ ]

Question 04 [Module 1: Parallel Computing Principles]

Question: Training a Random Forest where different CPU threads train independent decision trees on different subsets of the data represents which parallel computing paradigm?

- **a)** Instruction-Level Speculation
- **b)** Task Parallelism combined with Data Parallelism
- **c)** Serial Pipelining
- **d)** Lock-step SIMD Vectorization

Candidate Selection: [ ]

Question 05 [Module 1: Parallel Computing Principles]

Question: If a simulation runs in 100 seconds on 1 core and in 20 seconds on 8 cores, what is the Parallel Efficiency (E) of this scaling?

- **a)** 80%
- **b)** 62.5%
- **c)** 50%
- **d)** 25%

Candidate Selection: [ ]

Question 06 [Module 2: OpenMP for Multithreading]

Question: What happens during the transition from serial to parallel execution in the OpenMP Fork-Join programming model?

- **a)** The computer creates a new virtual machine
- **b)** The single initial Master Thread encounters an OpenMP parallel directive and forks a team of worker threads; at the end of the parallel region, worker threads synchronize at an implicit barrier and join back into the master thread
- **c)** The operating system reboots the CPU socket into hyper-threading mode
- **d)** All threads are terminated and written to disk swap

Candidate Selection: [ ]

Question 07 [Module 2: OpenMP for Multithreading]

Question: In `#pragma omp parallel for reduction(+:total_sum)`, how does the OpenMP compiler prevent data races when accumulating values across millions of loop iterations?

- **a)** It forces all threads to execute one at a time sequentially
- **b)** It allocates a private local copy of `total_sum` initialized to 0 for each thread, accumulates locally without synchronization during the loop, and atomically combines the private sums into the master variable at loop exit
- **c)** It turns off compiler optimizations so numbers add up slowly
- **d)** It writes each addition operation to a temporary text file on disk

Candidate Selection: [ ]

Question 08 [Module 2: OpenMP for Multithreading]

Question: What is the critical bug in the following OpenMP loop: `#pragma omp parallel for shared(idx) for (idx=0; idx<N; idx++) { process(idx); }`?

- **a)** The loop index `idx` is declared `shared`, causing all threads to concurrently overwrite the loop counter and corrupt iteration bounds
- **b)** The `process()` function cannot accept integer arguments
- **c)** OpenMP loops must always count downwards from N to 0
- **d)** C++ prohibits using `for` loops inside pragmas

Candidate Selection: [ ]

Question 09 [Module 2: OpenMP for Multithreading]

Question: You are parallelizing an algorithm where iteration `i` takes $O(i^2)$ time (each subsequent iteration takes exponentially more computation). Which OpenMP loop scheduling clause provides the best load balancing across threads?

- **a)** `schedule(static, 100)`
- **b)** `schedule(dynamic, 1)` or `schedule(guided)`
- **c)** `schedule(none)`
- **d)** `schedule(random)`

Candidate Selection: [ ]

Question 10 [Module 2: OpenMP for Multithreading]

Question: Which environment variable is read by the OpenMP runtime at program startup to set the default number of worker threads?

- **a)** `NUM_CPUS`
- **b)** `OMP_NUM_THREADS`
- **c)** `INTEL_MAX_THREADS`
- **d)** `THREAD_POOL_SIZE`

Candidate Selection: [ ]

Question 11 [Module 3: MPI for Distributed Systems]

Question: In the Message Passing Interface (MPI) standard, what is the role of `MPI_COMM_WORLD`?

- **a)** It is an internet web browser built into MPI
- **b)** It is the default global communicator containing all processes launched when `mpirun` or `mpiexec` was invoked
- **c)** It is the root IP address of the supercomputer login node
- **d)** It is a configuration file on the Linux hard drive

Candidate Selection: [ ]

Question 12 [Module 3: MPI for Distributed Systems]

Question: Which two fundamental MPI functions return the unique ID of the calling process and the total number of processes in the communicator respectively?

- **a)** `MPI_Get_id()` and `MPI_Get_total()`
- **b)** `MPI_Comm_rank()` and `MPI_Comm_size()`
- **c)** `MPI_Whoami()` and `MPI_Count()`
- **d)** `MPI_Process_rank()` and `MPI_Cluster_len()`

Candidate Selection: [ ]

Question 13 [Module 3: MPI for Distributed Systems]

Question: What 6 parameters compose the standard message envelope required by `MPI_Send(buf, count, datatype, dest, tag, comm)`?

- **a)** URL, port, username, password, token, encryption_key
- **b)** Memory buffer pointer, number of elements, MPI datatype, destination rank, message identification tag, and communicator
- **c)** Filename, line number, file size, permissions, owner, group
- **d)** CPU frequency, voltage, fan speed, core ID, socket ID, NUMA node

Candidate Selection: [ ]

Question 14 [Module 3: MPI for Distributed Systems]

Question: What is the key functional difference between `MPI_Bcast` and `MPI_Scatter` in distributed cluster computing?

- **a)** `MPI_Bcast` sends data only to Rank 0; `MPI_Scatter` sends data to all ranks
- **b)** `MPI_Bcast` copies the exact same data buffer identically to every process in the communicator; `MPI_Scatter` divides an array into equal segments and sends a distinct segment to each rank
- **c)** `MPI_Scatter` can only run on GPUs; `MPI_Bcast` only runs on CPUs
- **d)** `MPI_Bcast` deletes the data after sending

Candidate Selection: [ ]

Question 15 [Module 3: MPI for Distributed Systems]

Question: When computing the global loss across distributed nodes in deep learning or parallel ML, why is `MPI_Allreduce` used instead of `MPI_Reduce`?

- **a)** `MPI_Allreduce` performs the mathematical reduction across all processes and distributes the final reduced result to EVERY process; `MPI_Reduce` only leaves the final result on the designated root rank
- **b)** `MPI_Allreduce` operates without network cables
- **c)** `MPI_Reduce` is deprecated and deleted from the MPI standard
- **d)** `MPI_Allreduce` only supports integer data

Candidate Selection: [ ]

Question 16 [Module 4: Data Decomposition & Synchronization]

Question: In scientific grid simulations and 2D image processing, why is 2D domain decomposition (grid checkerboard) superior to 1D domain decomposition (slab slicing) at high processor counts?

- **a)** 2D decomposition reduces the surface-to-volume communication ratio, requiring significantly less boundary data to be exchanged over the network as core count increases
- **b)** 1D decomposition cannot handle floating-point numbers
- **c)** 2D decomposition eliminates the need for RAM
- **d)** 1D decomposition only works on Intel processors

Candidate Selection: [ ]

Question 17 [Module 4: Data Decomposition & Synchronization]

Question: What is the primary purpose of 'Halo' or 'Ghost' cells in distributed domain decomposition stencil computations?

- **a)** To hide secret passwords from other users on the supercomputer
- **b)** To replicate boundary memory layers from neighboring subdomains locally, enabling each process to compute stencil updates independently without querying remote memory during the compute phase
- **c)** To convert single-precision floats into double-precision floats automatically
- **d)** To store backup copies of the operating system kernel

Candidate Selection: [ ]

Question 18 [Module 4: Data Decomposition & Synchronization]

Question: Under what exact condition does a 'Data Race' occur in a multi-threaded OpenMP program?

- **a)** When two threads execute on different CPU sockets at different clock speeds
- **b)** When two or more concurrent threads access the exact same memory location without synchronization, and at least one of the accesses is a write operation
- **c)** When a thread takes longer than 1 millisecond to complete a loop
- **d)** When the compiler uses `-O3` optimization

Candidate Selection: [ ]

Question 19 [Module 4: Data Decomposition & Synchronization]

Question: Consider two MPI processes: Rank 0 executes `MPI_Recv(from Rank 1)` then `MPI_Send(to Rank 1)`. Simultaneously, Rank 1 executes `MPI_Recv(from Rank 0)` then `MPI_Send(to Rank 0)`. What happens?

- **a)** The data is transferred successfully in 1 millisecond
- **b)** Both processes enter an indefinite Deadlock, each blocked waiting for the other to send a message that cannot be sent until the receive completes
- **c)** MPI automatically terminates Rank 1 and restarts the job
- **d)** The operating system converts the blocking calls into non-blocking calls automatically

Candidate Selection: [ ]

Question 20 [Module 4: Data Decomposition & Synchronization]

Question: Why is `#pragma omp atomic` significantly faster than `#pragma omp critical` for updating a shared counter (`counter++`) in OpenMP?

- **a)** `#pragma omp atomic` turns off cache coherency across cores
- **b)** `#pragma omp atomic` leverages dedicated hardware CPU atomic instructions (e.g., `LOCK INC` or `CMPXCHG`), whereas `#pragma omp critical` establishes a heavyweight software mutex lock across all threads
- **c)** `#pragma omp critical` only works in C, while `atomic` works in C++
- **d)** `#pragma omp atomic` skips writing to physical RAM

Candidate Selection: [ ]

Question 21 [Module 5: Intel AI Analytics Toolkit & oneDAL Acceleration]

Question: What is the primary role of the Intel oneAPI Data Analytics Library (oneDAL) within the Intel AI Analytics Toolkit?

- **a)** To scrape websites for financial stock prices
- **b)** To provide highly optimized, hardware-accelerated algorithmic kernels for all data analysis stages (preprocessing, transformation, classical ML training and inference) leveraging AVX-512 and AMX on Intel architectures
- **c)** To convert Python code into Java bytecode
- **d)** To manage physical hard drive partitions

Candidate Selection: [ ]

Question 22 [Module 5: Intel AI Analytics Toolkit & oneDAL Acceleration]

Question: How can an existing Python machine learning project using Scikit-Learn achieve Intel oneDAL acceleration without modifying model training or prediction code?

- **a)** By rewriting all Python models in CUDA C++
- **b)** By adding `from sklearnex import patch_sklearn; patch_sklearn()` at the top of the Python script before importing any Scikit-learn modules
- **c)** By running `export FORCE_INTEL_GPU=1` in the Linux terminal
- **d)** By renaming all `.py` files to `.cpp`

Candidate Selection: [ ]

Question 23 [Module 5: Intel AI Analytics Toolkit & oneDAL Acceleration]

Question: Why does Intel Extension for Scikit-learn typically deliver 10x to 40x speedups on K-Means clustering benchmarks compared to standard Scikit-learn?

- **a)** It uses random guessing to pick cluster centers in zero time
- **b)** It utilizes vectorized AVX-512 distance computation kernels, cache-friendly data layouts, and parallelized Lloyd's algorithmic updates optimized for Intel Xeon memory hierarchies
- **c)** It limits clustering to a maximum of 2 clusters
- **d)** It downsamples the dataset by deleting 90% of the rows

Candidate Selection: [ ]

Question 24 [Module 5: Intel AI Analytics Toolkit & oneDAL Acceleration]

Question: Which computational bottleneck in classical Support Vector Machines (SVC) is accelerated by Intel oneDAL on Intel Xeon CPUs?

- **a)** HTML parsing and data visualization
- **b)** Sequential Minimal Optimization (SMO) kernel matrix evaluations and working set selection
- **c)** Audio decoding and speech synthesis
- **d)** Hard disk defragmentation

Candidate Selection: [ ]

Question 25 [Module 5: Intel AI Analytics Toolkit & oneDAL Acceleration]

Question: If a developer needs to unpatch Scikit-learn to compare empirical benchmark results against stock Scikit-learn in the same Python session, what function should they invoke?

- **a)** `sklearnex.reset_system()`
- **b)** `sklearnex.unpatch_sklearn()`
- **c)** `scipy.restore_defaults()`
- **d)** `sys.exit_intel()`

Candidate Selection: [ ]
